

# UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Peru, DECANA DE AMERICA  
FACULTAD DE INGENIERIA DE SISTEMAS E INFORMATICA  
Escuela Profesional de Ingenieria de Sistemas

## APLICACION BASADA EN MICROSERVICIOS TOMANDO COMO BASE EL BACKEND DEL PROYECTO DEL CURSO

### Informe

|             |                                                                                      |
|-------------|--------------------------------------------------------------------------------------|
| PROFESORA   | Mamani Rodriguez, Zoraida Emperatriz                                                 |
| CURSO       | Desarrollo de Sistemas Web                                                           |
| GRUPO       | 11                                                                                   |
| INTEGRANTES | Cotrina Ruiz, Angel Eduardo; Moya Laureano, Jussel Enrique; Tocto Caqui, Juan Andres |

Lima - Peru  
2026 - I

## Indice

1. Equipo del Proyecto
2. Arquitectura de Microservicios del Proyecto
3. Entregables del Proyecto
  - 3.1. Microservicios
  - 3.2. Frontend
4. Logros del Sprint
5. Pendientes del Sprint
6. Conclusiones
7. Recomendaciones
8. Anexos

## 1. Equipo del Proyecto

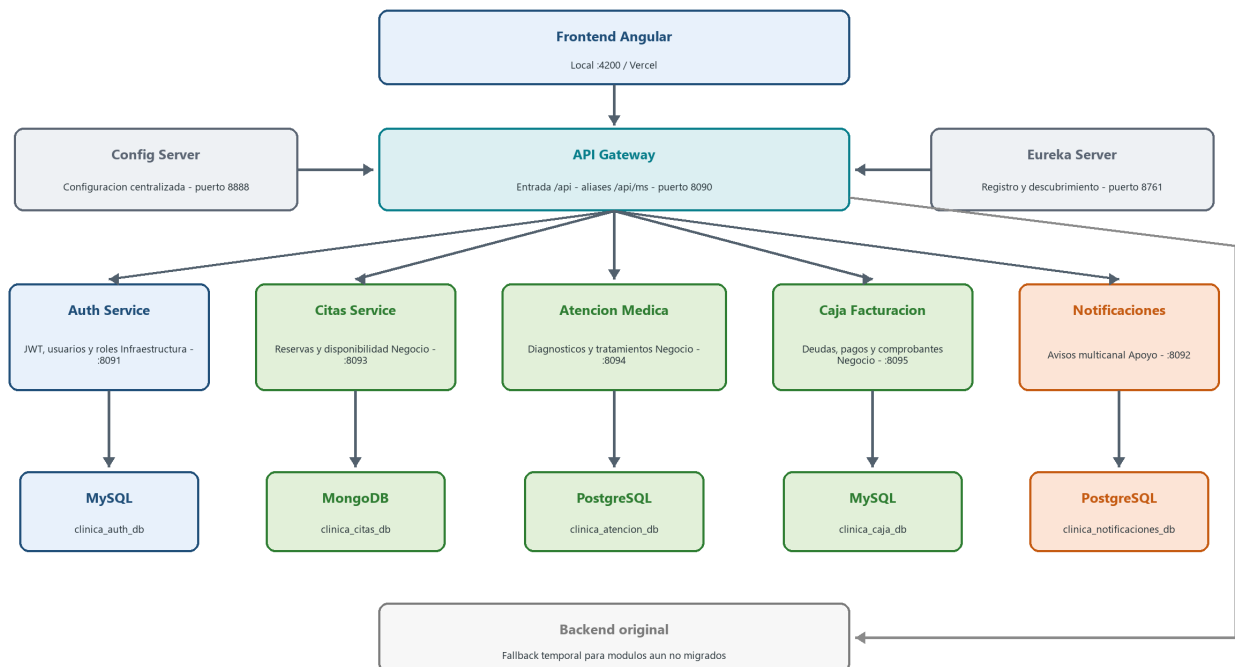
| Nombre                        | Rol                                  | Github         |
|-------------------------------|--------------------------------------|----------------|
| Cotrina Ruiz, Angel Eduardo   | Desarrollador principal / Full Stack | AngelCotrina28 |
| Moya Laureano, Jussel Enrique | Scrum Master / Full Stack            | Enrike352      |
| Tocto Caqui, Juan Andres      | Product Owner / Full Stack           | RexoJuan       |

## 2. Arquitectura de Microservicios del Proyecto

La propuesta aplica una migracion gradual. Angular utiliza el API Gateway como entrada unica; /api/ms selecciona los servicios nuevos y el fallback mantiene temporalmente los modulos heredados.

### Arquitectura de Microservicios - Clinica Privada

Entrada unica, servicios independientes y persistencia por dominio



| Componente       | Tipo            | Responsabilidad                         |
|------------------|-----------------|-----------------------------------------|
| Gateway          | Infraestructura | Entrada unica, JWT y enrutamiento.      |
| Config Server    | Infraestructura | Configuracion centralizada local.       |
| Eureka           | Infraestructura | Registro y descubrimiento local.        |
| Auth             | Infraestructura | Login, usuarios, roles y JWT.           |
| Citas            | Negocio         | Reservas, disponibilidad y cancelacion. |
| Atencion Medica  | Negocio         | Diagnosticos y tratamientos.            |
| Caja Facturacion | Negocio         | Deudas, pagos y comprobantes.           |
| Notificaciones   | Apoyo           | Avisos para pacientes y trabajadores.   |

## 3. Entregables del Proyecto

### 3.1. Microservicios

El repositorio contiene ocho aplicaciones Spring Boot independientes. Los tres servicios de negocio usan MongoDB, PostgreSQL y MySQL.

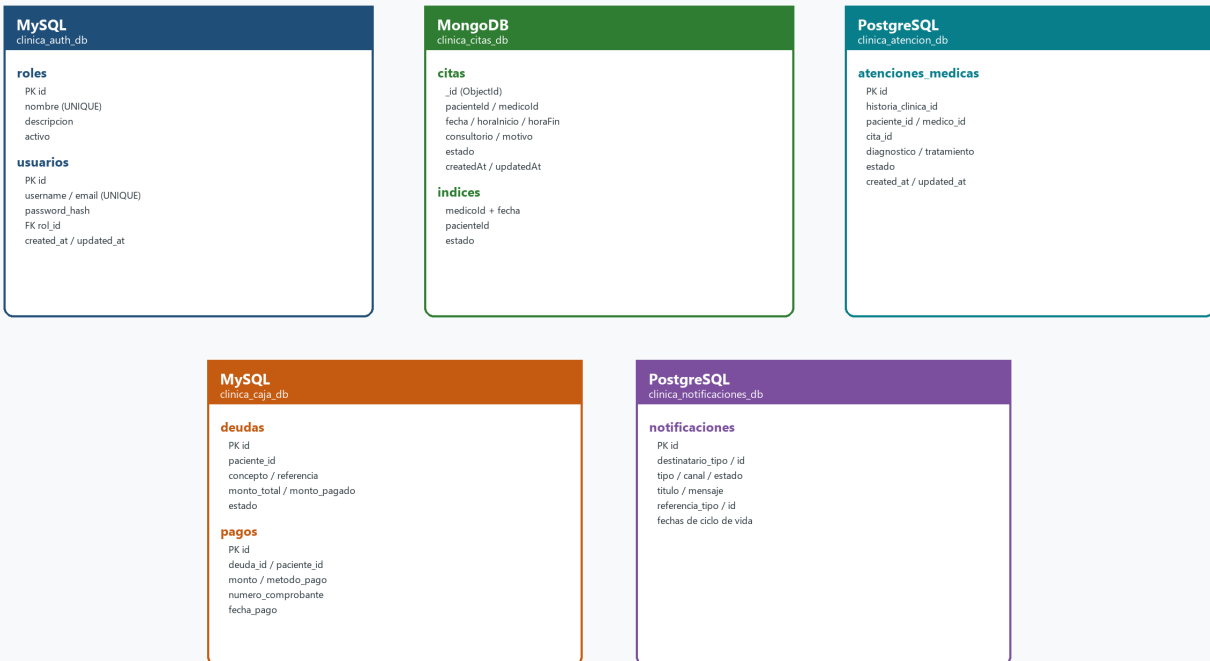
#### 3.1.1. URL del Repositorio Github

<https://github.com/AngelCotrina28/ClinicaPrivadaMSContainer>

#### 3.1.2. Esquemas de las bases de datos

##### Esquemas de Bases de Datos por Microservicio

Tres motores, cinco bases independientes y propiedad de datos por servicio



| Servicio       | Motor      | Base                      | Artefactos                         |
|----------------|------------|---------------------------|------------------------------------|
| Auth           | MySQL      | clinica_auth_db           | database/mysql/auth                |
| Citas          | MongoDB    | clinica_citas_db          | database/mongodb/citas             |
| Atencion       | PostgreSQL | clinica_atencion_db       | database/postgresql/atencion       |
| Caja           | MySQL      | clinica_caja_db           | database/mysql/caja                |
| Notificaciones | PostgreSQL | clinica_notificaciones_db | database/postgresql/notificaciones |

#### 3.1.3. Microservicios desplegados localmente

- Docker Compose mediante start-microservices.ps1.
- Kubernetes mediante start-k8s.ps1.
- Validacion integral mediante verify-microservices.ps1 y verify-k8s.ps1.

## Evidencia de validacion tecnica

Resultados obtenidos durante la implementacion

|                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Kubernetes local</b> <ul style="list-style-type: none"><li>• Gateway health: UP</li><li>• Login por Gateway: OK</li><li>• Notificacion creada: OK</li><li>• Cita creada: OK</li><li>• Atencion registrada y cerrada: OK</li><li>• Deuda y pago registrados: OK</li><li>• Cruce de horario bloqueado: OK</li><li>• Validacion completa: OK</li></ul> | <b>Gateway Service</b> <ul style="list-style-type: none"><li>• mvnw.cmd test</li><li>• Tests run: 3</li><li>• Failures: 0</li><li>• Errors: 0</li><li>• BUILD SUCCESS</li></ul>                                                                  |
| <b>Frontend Angular</b> <ul style="list-style-type: none"><li>• npm run build</li><li>• Environment API URL: /api</li><li>• Application bundle generation complete</li><li>• Salida: dist/frontend/browser</li></ul>                                                                                                                                   | <b>Artefactos verificables</b> <ul style="list-style-type: none"><li>• Docker Compose y Kubernetes</li><li>• Scripts PowerShell de arranque y prueba</li><li>• Coleccion Postman local y Render</li><li>• DDL/SQL/JS para tres motores</li></ul> |

### 3.1.4. URL de los Microservicios Desplegados en la nube

| Servicio       | URL                                           | Base               |
|----------------|-----------------------------------------------|--------------------|
| Gateway        | clinica-gateway-service.onrender.com          | No aplica          |
| Auth           | clinica-auth-service.onrender.com             | MySQL / Aiven      |
| Citas          | clinica-citas-service.onrender.com            | MongoDB Atlas      |
| Atencion       | clinica-atencion-medica-service.onrender.com  | PostgreSQL / Aiven |
| Caja           | clinica-caja-facturacion-service.onrender.com | MySQL / Aiven      |
| Notificaciones | clinica-notificaciones-service.onrender.com   | PostgreSQL / Aiven |

En el plan gratuito los servicios pueden entrar en reposo; antes de exponer se debe abrir /actuator/health y esperar status UP.

### 3.1.5. Coleccion de servicios definidos en Postman

postman/ contiene la coleccion Postman 2.1 y ambientes local/Render. El flujo almacena automaticamente JWT e identificadores y prueba Citas, Atencion, Caja y Notificaciones.

## 3.2. Frontend

### 3.2.1. URL del Repositorio Github

<https://github.com/AngelCotrina28/Clinica-Privada-Frontend>

Frontend desplegado: <https://clinica-privada-frontend.vercel.app/>

Angular conserva /api como URL unica. proxy.conf.json apunta al Gateway local :8090 y vercel.json al Gateway de Render.

## 4. Logros del Sprint

- Tres microservicios de negocio con tres motores distintos.

- Servicios de infraestructura y apoyo implementados.
- Docker Compose y Kubernetes locales reproducibles.
- Servicios desplegados en Render con bases de Aiven y MongoDB Atlas.
- JWT propagado por Gateway y frontend ajustado.
- DDL, datos de ejemplo y coleccion Postman versionados.
- Gateway: 3 pruebas, 0 fallos. Angular: build satisfactorio.

## 5. Pendientes del Sprint

Como trabajo evolutivo no bloqueante quedan la migracion total del backend original, CI/CD, observabilidad y un despliegue Kubernetes administrado en nube.

## 6. Conclusiones

- La separacion por dominios reduce acoplamiento y permite despliegues independientes.
- La persistencia poliglota asigna cada motor segun las necesidades del dominio.
- Docker Compose facilita el arranque y Kubernetes administra el estado declarado.
- El Gateway permite una migracion gradual sin reescribir Angular de una sola vez.

## 7. Recomendaciones

- Ejecutar verify-k8s.ps1 o Postman antes de cada demostracion.
- Mantener contraseñas en variables o Secrets, nunca en Git.
- Evitar acceso directo entre bases de datos de distintos servicios.
- Versionar los contratos mientras se retira el fallback heredado.

## 8. Anexos

### 8.1. Contribuciones obtenidas de Git

| Repositorio             | Cuenta                           | Commits |
|-------------------------|----------------------------------|---------|
| Microservices Container | AngelCotrina28                   | 16      |
| Frontend                | AngelCotrina28 / Angel Cotrina   | 42      |
| Frontend                | RexoJuan (Git: JuanTD / Juan TD) | 30      |
| Frontend                | Enrike352                        | 26      |
| Backend                 | AngelCotrina28 / Angel Cotrina   | 65      |
| Backend                 | RexoJuan (Git: JuanTD / Juan TD) | 20      |
| Backend                 | Enrike352                        | 24      |

### 8.2. Evidencias finales pendientes

- Capturas finales de Docker Desktop, Kubernetes, Render, Aiven y MongoDB Atlas para la version entregable y el video.